

Universidad del Valle de Guatemala · CC3091 Ingeniería de Software 2

Tarea 3: Pruebas Automatizadas

Pruebas unitarias automatizadas en **Kontrol**

Node.js 20 + Express

Vue 3 + Vite

Pinia

MongoDB + PostgreSQL

Integrantes: [Nombre 1] · [Nombre 2] · [Nombre 3] · [Nombre 4] · [Nombre 5]

Semestre II — 2026

1. Frameworks investigados — Backend (Node.js)

Framework	Descripción	ESM nativo	Mocking	Cobertura	Popularidad
Jest	El más usado (Meta). Todo incluido	Experimental	✓	✓	~25M/sem
Vitest	Moderno, del equipo Vite/Vue. API Jest-compatible	✓ Nativo	✓	✓ (v8)	~10M/sem
Mocha + Chai + Sinon	Stack clásico modular (3 librerías)	Parcial	Sinon aparte	nyc aparte	~7M/sem
node:test	Runner nativo de Node ≥ 20, cero dependencias	✓	Básico	Básica	nativo
AVA	Minimalista, tests concurrentes	✓	✗	✗	~300K/sem

Dato clave: nuestro backend es ESM puro ("type": "module") → el soporte ESM experimental de Jest es fricción directa.

1. Frameworks investigados — Frontend (Vue 3)

Opción	Descripción	Integración con Vite	Estado
Vitest + @vue/test-utils	Combinación oficial recomendada por el equipo de Vue (default de create-vue)	Nativa — reutiliza vite.config	Activo, en crecimiento
Jest + vue3-jest	Estándar de la era Vue 2 / webpack	Duplica configuración (babel + transformers)	Mantenimiento rezagado
Testing Library (Vue)	Capa sobre test-utils, enfoque "probar como usuario"	Vía Vitest	Complemento, no runner
Cypress / Playwright CT	Componentes en navegador real	Setup pesado	Excesivo para unit tests

Entornos DOM simulados

- **happy-dom**: rápido y ligero (el que usamos) · **jsdom**: más completo pero más lento.

2. Nuestra elección: Vitest (front + back)

Integración con nuestro stack

- Frontend ya usa Vite → los tests reutilizan `vite.config.js` (alias `@/`, plugin Vue) sin duplicar nada.
- Backend ESM puro → Vitest lo soporta **nativamente**; Jest requiere flags experimentales.
- Recomendado oficialmente por el equipo de Vue.

Rendimiento

- Transformación con esbuild + workers en paralelo.
- Watch mode tipo HMR: re-ejecuta solo los tests afectados.

Complementos: `@vue/test-utils` `happy-dom` `@vitest/coverage-v8`

Comunidad y documentación

- +10M descargas/semana; default en Vue, Nuxt, Astro, SvelteKit.
- Docs completas (`vitest.dev`), releases frecuentes.
- API compatible con Jest → curva de aprendizaje ≈ 0 .

Robustez ("baterías incluidas")

- Mocks, spies y fake timers integrados (`vi.*`).
- Cobertura v8 integrada.
- Un solo framework para todo el monorepo → CI simple.

3. Configuración — mínima por diseño

backend/vitest.config.js

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    environment: 'node',
    globals: true,
    include: ['tests/**/*.test.js']
  }
})
```

frontend/vitest.config.js

```
import { defineConfig, mergeConfig } from 'vitest/config'
import viteConfig from './vite.config.js'

export default mergeConfig(
  viteConfig, // hereda alias @/ y plugin Vue
  defineConfig({
    test: {
      environment: 'happy-dom',
      globals: true,
      include: ['tests/**/*.test.js']
    }
  })
)
```

Scripts en ambos package.json: `npm test` · `npm run test:watch` · `npm run test:coverage`

4. Ejemplo backend #1 — Middleware JWT (requireAuth)

```
it('acepta un token válido, adjunta el payload en req.user y llama next()', () => {
  const payload = { id_usuario: 7, email: 'ana@kontrol.gt', nombre_rol: 'admin' }
  const token = jwt.sign(payload, SECRET)
  const req = { headers: { authorization: `Bearer ${token}` } }
  const res = buildRes() // { status: vi.fn(), json: vi.fn() }
  const next = vi.fn()

  requireAuth(req, res, next)

  expect(next).toHaveBeenCalledTimes(1)
  expect(req.user).toMatchObject(payload)
})

it('responde 401 cuando el token está expirado', () => {
  const token = jwt.sign({ id_usuario: 7 }, SECRET, { expiresIn: '-1s' })
  /* ... */
  expect(res.status).toHaveBeenCalledWith(401)
})
```

5 escenarios: token válido · sin header · esquema no-Bearer · expirado · secreto incorrecto. Sin servidor ni base de datos: req/res simulados con `vi.fn()`.

4. Ejemplos backend #2 y #3 — Validación Zod y RBAC

validate.js (Zod)

```
it('reemplaza req.body con datos parseados', () => {
  const req = { body: { email: 'ana@kontrol.gt',
                        edad: '25' } }
  validate(schema)(req, res, next)

  expect(next).toHaveBeenCalledTimes(1)
  // coerción: '25' (string) → 25 (número)
  expect(req.body.edad).toBe(25)
})

it('responde 400 con lista de errores', () => {
  const req = { body: { email: 'no-es-correo',
                        edad: 15 } }

  /* ... */
  expect(payload.errors.map(e => e.field))
    .toEqual(['email', 'edad'])
})
```

requireRole.js (RBAC)

```
it('permite el paso con rol en la lista', () => {
  const req = { user: { nombre_rol: 'admin' } }
  requireRole('admin', 'gerente')(req, res, next)
  expect(next).toHaveBeenCalledTimes(1)
})

it('responde 403 con rol no permitido', () => {
  const req = { user: { nombre_rol: 'colaborador' } }
  requireRole('admin')(req, res, next)
  expect(res.status).toHaveBeenCalledWith(403)
})
```

+ 4.º ejemplo: `utils/chat.js` — sanitización de adjuntos, previews y serialización (10 tests).

4. Ejemplo frontend #1 — Store Pinia con fetch mockeado

```
it('loadEmpresas consume la API y auto-selecciona la primera empresa', async () => {
  const empresas = [{ id_empresa: 5, nombre: 'Gamma' }]
  vi.stubGlobal('fetch', vi.fn().mockResolvedValue({
    ok: true,
    json: async () => ({ data: empresas }),
  }))

  const store = useAuthStore()
  store.setToken('jwt-de-prueba')
  await store.loadEmpresas()

  expect(fetch).toHaveBeenCalledWith('/api/companies/my-companies',
    { headers: { Authorization: 'Bearer jwt-de-prueba' } })
  expect(store.idEmpresaActual).toBe(5)
})
```

7 tests del store: login/logout, persistencia en localStorage (real, provisto por happy-dom), getters de permisos y selección de empresa. La API se aísla con `vi.stubGlobal`.

4. Ejemplos frontend #2 y #3 — Componente y helpers

Button.vue (@vue/test-utils)

```
it('renderiza la etiqueta recibida', () => {
  const w = mount(Button,
    { props: { label: 'Guardar' } })
  expect(w.text()).toBe('Guardar')
})

it('emite click al hacer clic', async () => {
  await w.trigger('click')
  expect(w.emitted('click')).toBeTruthy()
})
```

🐛 Bug real encontrado y corregido: sin emits: ['click'] el evento se duplicaba (nativo + \$emit). El test quedó como prueba de regresión.

statusHelpers.js (fake timers)

```
it('detecta tareas vencidas', () => {
  vi.useFakeTimers()
  vi.setSystemTime(new Date('2026-07-17'))

  expect(isOverdue({
    fecha_vencimiento: '2026-07-01',
    estado: 'EN_PROGRESO' })).toBe(true)
})

it('abrevia millones y miles', () => {
  expect(formatMoney(2_500_000)).toBe('2.5M')
  expect(formatBudget(4_500)).toBe('$5K')
})
```

+ 4.º ejemplo: utils/invitation.js — tokens y mensajes de error (7 tests).

5. Resultados de la ejecución

53

pruebas unitarias
(22 backend + 31 frontend)

< 1 s

duración por suite
(0.4 s back · 0.7 s front)

~80 %

cobertura statements
en archivos testeados

1

bug real detectado y corregido
(emits duplicado en Button)

Test Files 4 passed (4)
Tests 22 passed (22)
Duration 338ms ← *backend*

Test Files 4 passed (4)
Tests 31 passed (31)
Duration 663ms ← *frontend*

Tiempo invertido: ~15 min de setup + ~1.5 h escribiendo las 8 suites ≈ **1 h 45 min en total.**

6. Conclusiones

- **Configuración casi nula:** 8 líneas de config en backend; el frontend hereda `vite.config.js` completo (alias, plugin Vue) con `mergeConfig`.
- **Velocidad que cambia el flujo de trabajo:** suites completas en <1 s y watch mode instantáneo hacen viable correr los tests en cada guardado, no solo en CI.
- **Mocking integrado suficiente:** `vi.fn()` (req/res de Express), `vi.stubGlobal(fetch)` y `vi.useFakeTimers` (fechas) cubrieron todos nuestros casos sin librerías extra.
- **Los tests encontraron y ayudaron a corregir un bug real** el primer día: el evento click duplicado en `Button.vue` por no declarar `emits`; el test quedó como prueba de regresión.
- **Un solo framework para el monorepo** reduce carga cognitiva: mismos comandos y API en backend y frontend.
- **Contras honestos:** comunidad más joven que Jest y menor ecosistema de plugins; la ventaja se reduce en proyectos sin Vite.

Veredicto: Vitest es la elección correcta para Kontrol — en ~2 horas obtuvimos 53 pruebas listas para CI que protegen autenticación, autorización, validación y lógica de negocio.